Assignment: Planning in an LQR Environment with CEM and MPPI

Why this assignment is important

Many model-based reinforcement learning algorithms use the following pipeline:

$$\text{current state} \rightarrow \text{model rollout} \rightarrow \text{trajectory evaluation} \rightarrow \text{action selection}$$

This is closely related to **Model Predictive Control**, or MPC.

At every timestep, MPC does the following:

1. starts from the current state,
2. plans a sequence of future actions,
3. evaluates the predicted trajectory,
4. executes only the first action,
5. observes the next state,
6. replans again.

This is used in many MBRL methods, such as PETS, MBPO-style planning, TD-MPC, TD-MPC2, and MPPI-based robot control.

The assignment help our cute new members understand:

1. how MPC works,
2. how CEM and MPPI optimize action sequences,
3. why planning horizon matters,
4. when a terminal value function matters and when it does not.

Environment

The students are given an LQR environment.

The dynamics are:

$$s_{t+1} = As_t + Ba_t$$

or, in the stochastic setting,

$$s_{t+1} = As_t + Ba_t + \epsilon_t.$$

The reward is:

$$r(s_t, a_t) = -\left(s_t^\top Q s_t + a_t^\top R a_t\right).$$

Phase I — Implement MPC with Finite-Horizon Random Shooting

Recall that at every timestep, MPC does the following:

1. starts from the current state,
2. plans a sequence of future actions with length H,
3. evaluates the predicted trajectory,
4. executes only the first action,
5. observes the next state,
6. replans again.

Please implement an MPC algorithm with the given pseudocode. For Line 2., please do random shooting, i.e., initialize any distribution you like, and sample multiple action sequences from it.

Question Before you Write the Code

How does planning horizon affect performance? Why can short-horizon planning fail even when the dynamics are known exactly?

Experiment

Please try the following planning horizons:

$$H \in \{1, 2, \dots, 40\}$$

and plot a graph for episode (truncate at 10,000 steps) reward vs. planning horizon .

Phase II — Implement CEM and MPPI

In Phase I, random shooting samples every action sequence from a **fixed** proposal distribution and simply keeps the best one. Most of the samples are wasted in low-reward regions. CEM and MPPI fix this by **refining the sampling distribution** across iterations so that samples concentrate where the high-return action sequences actually live.

Both are still used as the inner loop of MPC: at every timestep we replan from the current state, execute only the first action, and **warm start** the next timestep with the (shifted) solution we just found.

CEM (Cross-Entropy Method)

CEM keeps a Gaussian over action sequences. Each iteration it samples from the Gaussian, keeps the top-K **elites** (highest return), and **refits both the mean and the std** of the Gaussian to those elites. The distribution collapses onto the best region over a few iterations.

```
Input: state s, horizon H, samples N, elites K (K<N),
       max_iters I, tolerances eps_mu (mean) and eps_sigma (std),
       warm-started mean mu (H x adim), initial std sigma (H x adim)

i = 0
```

```
repeat:
    mu_prev = mu

    # 1. sample N action sequences from current Gaussian
    for n = 1 ... N:
        a^(n) ~ N(mu, sigma^2)            # shape H x adim
        a^(n) = clip(a^(n), a_low, a_high)

    # 2. evaluate each sequence with the known model
    for n = 1 ... N:
        s_hat = s ; R^(n) = 0
        for h = 0 ... H-1:
            R^(n) += gamma^h * r(s_hat, a^(n)_h)
            s_hat  = A s_hat + B a^(n)_h

    # 3. pick the top-K elites by return
    E = indices of the K largest R^(n)

    # 4. refit BOTH mean and std to the elites
    mu    = mean_{n in E} a^(n)
    sigma = std_{n in E}  a^(n)

    i = i + 1
until  ||mu - mu_prev|| < eps_mu   OR   max(sigma) < eps_sigma   OR   i >= I

execute a_0 = mu_0
warm start: mu <- [mu_1, ..., mu_{H-1}, 0],  reset sigma for next timestep
```

Convergence: stop when the mean stops moving ($\|\mu - \mu_{prev}\| < \varepsilon_\mu$), **or** the distribution has collapsed ($\max \sigma < \varepsilon_\sigma$), **or** the iteration budget I is reached.

MPPI (Model Predictive Path Integral)

MPPI keeps a single **nominal** control sequence μ. Each iteration it samples N noisy perturbations of the nominal, scores them, and updates the nominal with a **softmax (reward-weighted) average** of all samples — nothing is thrown away. The temperature λ controls softness: $\lambda \to 0$ is greedy (only the best matters), $\lambda \to \infty$ is a plain average.

```
Input: state s, horizon H, samples N, temperature lambda, noise std sigma,
       max_iters I, tolerance eps (first-action mean change),
       warm-started nominal control sequence mu (H x adim)

i = 0
repeat:
    a0_prev = mu_0

    # 1. sample N sequences as noisy perturbations of the nominal
    for n = 1 ... N:
        eps^(n) ~ N(0, sigma^2)          # shape H x adim
        a^(n) = clip(mu + eps^(n), a_low, a_high)

    # 2. evaluate each sequence with the known model
```

```
    for n = 1 ... N:
        s_hat = s ; R^(n) = 0
        for h = 0 ... H-1:
            R^(n) += gamma^h * r(s_hat, a^(n)_h)
            s_hat  = A s_hat + B a^(n)_h

    # 3. softmax (exponential) weights over returns
    beta  = max_n R^(n)                    # baseline for numerical stability
    w^(n) = exp((R^(n) - beta) / lambda)
    w^(n) = w^(n) / sum_m w^(m)            # normalize, sum w = 1

    # 4. update nominal by reward-weighted average
    mu = sum_n w^(n) * a^(n)

    i = i + 1
until  ||mu_0 - a0_prev|| < eps   OR   i >= I

execute a_0 = mu_0
warm start: mu <- [mu_1, ..., mu_{H-1}, 0] for next timestep
```

Convergence: stop when the first action's mean stops moving ($\|\mu_0 - \mu_0^{prev}\| < \varepsilon$) — since only a_0 is actually executed, its stability means the plan has stabilized — **or** the iteration budget I is reached.

One-line difference: **CEM = refit mean+std to top-K elites; MPPI = exp-weighted average update of the mean.**

Experiment

Please try the following planning horizons:

$$H \in \{1, 2, \dots, 40\}$$

and plot a graph for episode (truncate at 10,000 steps) reward vs. planning horizon .

Phase III — Add the terminal value function

Now modify the objective to include the provided terminal value function:

$$J = \sum_{t=0}^{H-1} r(s_t, a_t) + V(s_H).$$

Repeat the experiments from Phase II.

Compare four methods:

1. CEM-MPC without terminal value
2. CEM-MPC with terminal value
3. MPPI-MPC without terminal value
4. MPPI-MPC with terminal value

Use the same planning horizons:

$$H \in \{1, 2, \dots, 40\}$$

Plot episode cost vs. planning horizon for all methods.

Note: The optimal value function of a given state can be calculated by the following program:

```
P = ctrl.P                                    # 來自 LQRController -> DARE 解
def terminal_value(states):                   # states: (N, 3)
    return -np.einsum("ni,ij,nj->n", states, P, states)   # 回傳 (N,)
```

Question

When does the terminal value function help the most? Is it more useful for short horizons or long horizons? Explain why.